

PROJECT REPORT
ProjectDock
A Modern Project Management & Team Collaboration
Platform

Submitted in partial fulfillment of the requirements for the award
of degree of

B.Tech (Computer Science and Engineering)

Submitted to

LOVELY PROFESSIONAL UNIVERSITY
PHAGWARA, PUNJAB



L OVELY
P ROFESSIONAL
U NIVERSITY

Submitted By:

Vikash Kumar, Sunny Kumar
Ajay Razz, Rohit Gussain

Submitted to:

Vishal Rathee
UID: 268608

Table of Contents

1. Executive Summary.....	3
2. Introduction.....	3
3. System Overview	4
4. Technology Stack.....	4
5. Project Structure	6
6. Core Features	6
7. Database Schema.....	10
8. API Overview	11
9. Role-Based Access Control.....	13
10. Local Development Setup.....	13
11. Environment Variables	15
12. Docker Deployment	15
13. Cloud Deployment.....	16
14. Security Considerations	17
15. Testing and Quality Assurance	17
16. Risk Assessment and Limitations.....	18
17. Glossary of Terms.....	18
18. Conclusion and Future Scope.....	19
Appendix A: Complete Feature List.....	20
References.....	20

1. Executive Summary

ProjectDock is a full-stack, production-ready project management platform designed to support software teams across the complete development lifecycle — from task creation and sprint planning through team collaboration, time tracking, and reporting. The platform draws direct inspiration from established tools such as Jira, ClickUp, Asana, and Linear, while delivering its own modern, responsive interface with real-time capabilities built on WebSockets.

This report documents the architecture, feature set, technology stack, data model, API surface, security approach, and deployment strategy of ProjectDock. It is intended to give a reader a complete and structured understanding of the system without requiring direct access to the source code, and to serve as a reference for onboarding, evaluation, or further development.

At a high level, ProjectDock is organized as a monorepo containing an independent backend and frontend. The backend is a Node.js REST API backed by a PostgreSQL database managed through the Prisma ORM. The frontend is a React single-page application built with Vite. The two halves of the system communicate over standard HTTP REST calls and a persistent WebSocket connection powered by Socket.io, which keeps every connected client synchronized the moment data changes.

The remainder of this report expands on each of these areas in turn: the system's overall architecture, the specific technologies chosen for the frontend and backend, the features available to end users, the underlying database schema, the API routes that expose those features, the role-based access control model that governs permissions, and the setup and deployment paths available for both local development and production hosting.

2. Introduction

2.1 Purpose of the Document

This document provides a structured technical overview of ProjectDock for readers who need to understand what the platform does, how it is built, and how it can be run, extended, or evaluated. It consolidates architecture notes, feature descriptions, schema definitions, and deployment instructions into a single reference.

2.2 Project Background

Modern software teams typically rely on a combination of separate tools to manage their work: a task tracker, a sprint planning board, a wiki, a chat application, and a time-tracking tool. ProjectDock was built to consolidate these overlapping responsibilities into one coherent platform, reducing the friction of switching between tools and keeping project context — tasks, conversations, documentation, and metrics — in one place.

2.3 Scope

The scope of ProjectDock covers project and task management, sprint planning, a Gantt-style timeline view, team management with real-time chat, project-scoped wikis and discussion boards, file attachments, time logging, organization-level analytics, in-app notifications, tokenized invitations, and

both native and Google OAuth authentication. Access to every feature is governed by a role-based permission system enforced at both the API and UI layers.

3. System Overview

3.1 Architecture Summary

ProjectDock follows a classic three-tier architecture: a React-based presentation layer, an Express-based application layer, and a PostgreSQL-based data layer. The frontend never talks to the database directly; every read or write passes through the backend's REST API, which enforces authentication and authorization before touching the database through Prisma.

In addition to the request-response REST layer, the backend runs a Socket.io server. When a task is updated, a comment is posted, or a chat message is sent, the backend emits an event over the WebSocket connection so that every other client currently viewing that project or team sees the change immediately, without polling or manually refreshing the page.

3.2 Monorepo Structure

The codebase is organized as a monorepo with two top-level applications and a shared orchestration file:

- `backend/` — the Express API, Prisma schema, controllers, middleware, and utility modules.
- `frontend/` — the React application, including pages, reusable components, layouts, and context providers.
- `docker-compose.yml` — orchestrates both containers for local or production deployment.

Within the backend, responsibilities are split cleanly across controllers (route handler logic), middleware (authentication, role guards, and error handling), routes (Express router definitions), and utilities (the database client, the Socket.io setup, the mailer, and the activity logger). Within the frontend, the structure separates reusable components, React context for authentication and sockets, page-level layouts, full page views, and helper utilities such as the shared Axios instance.

3.3 Communication Model

Two channels connect the frontend and backend. The first is conventional HTTP REST, authenticated with a JSON Web Token attached to each request, used for standard create, read, update, and delete operations. The second is a persistent Socket.io WebSocket connection, established once a user logs in, used exclusively for pushing real-time events — task changes, notifications, and chat messages — from the server to connected clients.

4. Technology Stack

ProjectDock's technology choices favor a modern, well-supported JavaScript ecosystem across both the frontend and backend, prioritizing developer productivity, real-time responsiveness, and a strongly typed data layer.

4.1 Frontend Stack

Technology	Purpose
React 19	Core UI component framework powering every view in the application.
Vite 8	Build tooling and local development server, chosen for near-instant reload speed.
Tailwind CSS v4	Utility-first styling used consistently across all components.
Framer Motion	Page transitions and micro-animations throughout the interface.
TanStack React Query v5	Server state management, caching, and background synchronization.
React Router v7	Client-side routing between all pages.
Socket.io Client	Maintains the real-time WebSocket connection to the backend.
Axios	HTTP client used for all REST API calls.
Chart.js + react-chartjs-2	Renders analytics charts on the dashboard and reporting pages.
React Quill	Rich text editor used for the project Wiki.
React Hook Form	Form state management across the application.
Lucide React	Icon library used throughout the UI.
React Hot Toast	Toast notifications for user feedback.

4.2 Backend Stack

Technology	Purpose
Node.js + Express 5	REST API server and routing layer.
Prisma ORM	Type-safe database access and schema migrations.
PostgreSQL	Primary relational database.
Socket.io	Real-time, bidirectional communication with connected clients.
JSON Web Tokens (JWT)	Stateless authentication for API requests.
bcrypt	One-way hashing of user passwords before storage.
Google Auth Library	Verifies Google OAuth tokens for social sign-in.
Cloudinary + Multer	File upload handling and cloud-based storage of attachments.
Nodemailer	Sends transactional emails, including invitations.
Helmet	Sets secure HTTP response headers.
Morgan	Logs incoming HTTP requests during development.
Compression	Compresses HTTP responses to reduce payload size.

5. Project Structure

The repository is laid out as follows:

```
Project Management SaaS/
├── backend/
│   ├── prisma/schema.prisma      # Full database schema
│   ├── src/controllers/          # Route handler logic
│   ├── src/middleware/           # Auth, role guards, error handler
│   ├── src/routes/               # Express router definitions
│   ├── src/utills/               # DB client, socket, mailer, activity logger
│   └── server.js                  # App entry point
├── frontend/
│   ├── src/components/           # Reusable UI components
│   ├── src/context/              # Auth and Socket React contexts
│   ├── src/layouts/              # DashboardLayout with Sidebar and Topbar
│   ├── src/pages/                # Full page views
│   └── src/utills/               # Axios instance, helpers
└── docker-compose.yml            # Orchestrates both containers
```

This separation keeps backend and frontend concerns fully independent — each has its own dependency tree, its own Dockerfile, and can be deployed to different hosting providers without affecting the other, as described later in the deployment section of this report.

6. Core Features

This section walks through every major feature area of ProjectDock in detail, describing both what each feature does and how it fits into the overall workflow of a software team.

6.1 Project Management

Every project in ProjectDock moves through a defined lifecycle of statuses: Active, On Hold, Completed, and Archived. Beyond status, each project carries a rich set of details — name, key, description, logo, banner, color theme, priority, visibility, tags, and an estimated budget — giving teams enough context to manage a project without leaving the platform.

A project dashboard surfaces at-a-glance cards showing total tasks, completed tasks, pending tasks, member count, and overall progress, so that anyone opening a project immediately understands its current state. Users can also favorite or bookmark projects they work in most often for quicker access from the sidebar.

The project key acts as a short, unique identifier used throughout the interface to reference a project compactly — much like the project prefixes used in tools such as Jira — which keeps task references, notifications, and search results readable even as the number of projects in an organization grows. Visibility settings determine whether a project is discoverable only to invited members or more broadly within the organization, giving administrators fine control over information boundaries between teams working on sensitive or client-facing engagements.

6.2 Task Management

Tasks are the atomic unit of work in ProjectDock. Each task can be created, updated, and deleted with rich metadata: title, description, priority (Low, Medium, High, or Critical), status, one or more

assignees, labels, a due date, and estimated hours. A drag-and-drop Kanban board displays tasks across fully customizable columns and swimlanes, letting teams model whatever workflow suits them.

Opening a task reveals a detail view with threaded comments, file attachments, and a complete activity history, so that the reasoning behind a task's current state is never lost. Beyond any single project, a global task list spans every project a user has access to, with filtering by assignee, priority, status, label, and due date.

Because columns and swimlanes are fully customizable per project, teams are not locked into a single fixed workflow. A support-oriented team might model columns around triage, investigation, and resolution, while a product team might prefer the more familiar To Do, In Progress, In Review, and Done. Swimlanes add a second grouping axis — commonly used to separate work by epic, priority, or individual contributor — so that a single board can convey both the horizontal flow of a task and a meaningful vertical grouping at the same time.

6.3 Sprint Planning

Sprints are time-boxed iterations with defined start and end dates. Tasks are assigned to a sprint, and the platform tracks sprint velocity as work completes. A dedicated sprint board view shows tasks progressing through columns specific to that sprint. When a sprint reaches its end date, a completion workflow automatically carries over any unfinished tasks into the next sprint, so nothing is silently dropped.

Sprint velocity, tracked automatically as tasks move to a completed state, gives teams a data-driven basis for planning future sprints rather than relying purely on estimation. Over several sprints, this creates a historical record that a team can use to judge whether its planned workload for an upcoming sprint is realistic given its recent throughput, which is particularly valuable during retrospectives.

6.4 Timeline (Gantt View)

For teams that need to see how tasks relate to each other over time, ProjectDock offers a visual Gantt-style timeline. It lays out task durations, dependencies, and progress directly on a horizontal calendar, giving project managers a single view of how the whole project is paced against its deadlines.

The timeline view is particularly useful for identifying scheduling conflicts before they become blockers — for instance, spotting that two dependent tasks have been assigned overlapping windows, or that a downstream task's start date has slipped past the completion date of the work it depends on. Because the timeline draws from the same underlying task data as the Kanban board, updates made in one view are reflected immediately in the other.

6.5 Team Management

Teams can be created and managed within an organization, each with an assigned team lead and a roster of individual members holding specific roles. A dedicated team detail page brings together a member directory, an activity feed, real-time team chat, and file storage scoped to that team.

Modeling teams as first-class entities, separate from projects, reflects how software organizations are typically structured in practice: a stable team of engineers may rotate across several projects over time,

and keeping the team's membership, chat history, and shared files independent of any single project avoids fragmenting that continuity every time the team's project assignments change.

6.6 Project Members and Access Control

Individual users can be added directly to a project with a specific role, or an entire team can be assigned to a project in a single action, saving repetitive setup work when a whole team moves onto a new engagement. Role-based permission checks run at both the API and UI level, so that users only ever see and interact with the parts of the system they are authorized to access.

6.7 Analytics and Reporting

An organization-level analytics dashboard visualizes task completion rates, sprint performance, team workload, and overall project health using interactive line, bar, and donut charts built with Chart.js. The dashboard also includes a time-logged-per-user breakdown and dedicated overdue task reporting, giving managers a quick way to spot bottlenecks.

Because analytics are computed from the same live data that powers the task boards and timesheets, the numbers a manager sees on the dashboard are never a stale export — they reflect the current state of every project the moment the dashboard is opened. This makes the analytics view suitable for use in daily stand-ups or weekly status meetings without any manual report preparation beforehand.

6.8 Timesheet and Time Logging

From the Timesheet page, users can start and stop a live timer against any task, or log hours manually after the fact for work that wasn't tracked in real time. Daily and weekly summaries roll logged time up per project, supporting both personal time management and project-level billing or capacity reviews.

6.9 Collaboration Tools

Each project includes a scoped Wiki, built on a rich text editor, for documenting decisions, guides, and architecture notes in one place tied directly to the project it describes. Alongside the Wiki, project Discussions provide forum-style threads with nested replies for asynchronous team communication, and a Project Files area handles attachments scoped to the project.

Keeping documentation, discussion, and files scoped to the project that produced them — rather than in a separate, general-purpose wiki or chat tool — means that context is never more than a click away from the tasks it relates to. A new team member joining a project can review its wiki and discussion history in place, without needing to be pointed to a separate system or search through an unrelated company-wide knowledge base.

6.10 Notifications

Real-time in-app notifications alert users to key events — task assignments, team additions, project invitations, and comment mentions — the moment they happen. A notification badge in the topbar opens a dropdown where users can review and clear notifications without leaving their current page.

6.11 Invitations

Organizations and projects can invite new members through a secure, tokenized email link. The accept-invite flow handles both brand-new users, who are guided through account creation, and existing users, who are added directly, keeping onboarding friction low regardless of whether the invitee already has an account.

6.12 User Settings and Profile

From the Settings page, users can update their profile information, avatar, and password. A light and dark mode toggle is available throughout the application, with the chosen preference persisted so it's remembered on the next visit.

6.13 Authentication

ProjectDock supports native email and password authentication, with passwords hashed using bcrypt before they are ever stored, alongside Google OAuth 2.0 sign-in through the official Google Identity Services library. Successful authentication issues a JWT, which is stored client-side and attached to every subsequent API request to establish the user's session.

7. Database Schema

The Prisma schema defines the relational data model that underlies every feature described in the previous section. The table below summarizes each core model and its role in the system.

Model	Description
User	Platform user with profile, role, and OAuth credentials.
Organization	Top-level workspace that owns teams and projects.
OrganizationMember	Join table linking users to organizations with a role.
Team	A named group of users within an organization.
TeamMember	Join table linking users to teams with a role and designation.
Project	A work container with settings, budget, and status.
ProjectMember	Join table linking users to projects with a role.
Task	Atomic unit of work with priority, status, assignees, and metadata.
TaskAssignee	Join table for multi-user task assignment.
Sprint	A time-boxed iteration of tasks within a project.
BoardColumn	Customizable Kanban columns per project.
Swimlane	Horizontal grouping rows on the Kanban board.
WikiPage	Rich text documentation page scoped to a project.
Discussion	A threaded forum post within a project.
DiscussionReply	A reply to a discussion thread.
Comment	Inline comment on a task.
Attachment	File metadata linked to a task, project, or team.
TimeLog	A work log entry linking a user, task, and duration.
Notification	In-app notification record per user.
ActivityLog	Immutable audit trail of user actions.
Invitation	Tokenized invite record with expiry.
TeamMessage	Real-time chat message within a team.

Together these models capture the full graph of a project's activity: who belongs to what organization, team, and project; what work exists and who owns it; how that work is grouped into sprints and board columns; and every comment, file, log entry, and message generated along the way.

8. API Overview

The backend exposes a REST API under the /api prefix. With the exception of a small number of public authentication routes, every endpoint is protected by JWT authentication middleware and, where relevant, role-based authorization checks.

Route Prefix	Description
/api/auth	Register, login, Google OAuth, get current user.
/api/users	User profile management.
/api/projects	Project CRUD, member management, team assignment.
/api/tasks	Task CRUD, assignments, comments, attachments.
/api/teams	Team CRUD, member management, messaging, files.
/api/sprints	Sprint management and task assignment.
/api/time-logs	Time entry creation and retrieval.
/api/analytics	Aggregated stats and chart data.
/api/notifications	Fetch and mark notifications as read.
/api/activity	Activity log retrieval.
/api/wiki	Wiki page CRUD per project.
/api/discussions	Discussion and reply management per project.
/api/search	Global search across projects, tasks, and users.
/api/invites	Invite generation and token acceptance.
/api/upload	File upload endpoint (Cloudinary).

8.1 Example Request — Creating a Task

The following illustrates the shape of a typical authenticated request against the task API. The client sends a POST request to /api/tasks with a JSON body describing the new task, and includes the JWT obtained at login in the Authorization header.

```
POST /api/tasks
Authorization: Bearer <jwt>
Content-Type: application/json
```

```
{
  "title": "Fix IDOR vulnerability in task endpoint",
  "projectId": "proj_8f2a",
  "priority": "HIGH",
  "status": "TODO",
  "assigneeIds": ["user_10"],
  "dueDate": "2026-08-20"
}
```

On success, the API responds with the newly created task record, including its generated identifier and timestamps, and the backend emits a task:created event over Socket.io to every client currently subscribed to that project's board so the new card appears immediately without a manual refresh.

8.2 Error Handling Conventions

The API follows a consistent error response shape across all routes: a JSON body containing a message field describing the failure and, where applicable, a field-level errors array for validation failures. Authentication failures return HTTP 401, authorization failures return HTTP 403, and validation failures return HTTP 422, allowing frontend error handling to branch on status code rather than parsing message text.

9. Role-Based Access Control

9.1 Role Enum

ProjectDock uses a single Role enum applied consistently across every membership scope — organization, team, and project. This keeps the permission model easy to reason about, since a role always means the same thing no matter where it is applied. In descending order of overall access, the roles are:

- `SUPER_ADMIN` — full platform-wide access.
- `ORG_ADMIN` — full control within an organization.
- `PROJECT_MANAGER` — manages projects, sprints, and members.
- `TEAM_LEAD` — manages a team and its assignments.
- `DEVELOPER` — creates and updates tasks and content.
- `QA_TESTER` — reviews and verifies task completion.
- `CLIENT` — limited external visibility into project status.
- `VIEWER` — read-only access.

9.2 Middleware Layers

Every protected API route runs through two middleware layers in sequence. The first, `protect`, verifies the JWT included in the request and attaches the authenticated user to it. The second, `authorizeRoles(...roles)`, checks that the authenticated user's role appears in the list of roles permitted for that specific operation. For example, only users holding `SUPER_ADMIN`, `ORG_ADMIN`, or `PROJECT_MANAGER` can delete a project, while a `VIEWER` can read project data but cannot create tasks or modify membership.

9.3 Frontend Permission Gating

On the client side, a `PermissionGate` component wraps UI elements that correspond to destructive or administrative actions. It checks the current user's role before rendering its children, ensuring that buttons or menu items for actions a user isn't authorized to perform are never shown in the first place, rather than being shown and then rejected by the API.

10. Local Development Setup

10.1 Prerequisites

- Node.js 18 or later
- npm 9 or later
- A PostgreSQL database, local or cloud-hosted via Neon or Supabase
- A Cloudinary account for file uploads (the free tier is sufficient)

10.2 Setup Steps

Step 1 — Clone the repository:

```
git clone https://github.com/Vikashh08/Project-Management-SaaS.git  
cd "Project Management SaaS"
```

Step 2 — Install backend dependencies, configure environment variables (see Section 11), then run database migrations and start the server:

```
cd backend  
npm install  
npx prisma db push  
npm run dev
```

The API becomes available at <http://localhost:5001>.

Step 3 — In a new terminal, install and start the frontend:

```
cd frontend  
npm install  
npm run dev
```

The application becomes available at <http://localhost:5173>.

11. Environment Variables

The backend requires a `.env` file with the following variables configured before it will start correctly:

Variable	Purpose
PORT	Port the backend server listens on (default 5001).
NODE_ENV	Runtime environment, e.g. development or production.
DATABASE_URL	PostgreSQL connection string used by Prisma.
JWT_SECRET	Secret used to sign and verify JSON Web Tokens.
GOOGLE_CLIENT_ID	OAuth client ID for Google sign-in.
CLOUDINARY_CLOUD_NAME	Cloudinary account identifier.
CLOUDINARY_API_KEY	Cloudinary API key for uploads.
CLOUDINARY_API_SECRET	Cloudinary API secret for uploads.
EMAIL_HOST / EMAIL_PORT	SMTP host and port for outgoing email.
EMAIL_USER / EMAIL_PASS	SMTP credentials.
FROM_EMAIL / FROM_NAME	Sender identity used on outgoing emails.
FRONTEND_URL	Base URL used to build invitation links.

12. Docker Deployment

ProjectDock is fully containerized and can be brought up with a single Docker Compose command, provided Docker Desktop is installed and running:

```
docker compose up -d --build
```

This single command performs three actions:

- Builds the backend Node.js image and generates the Prisma client inside the container.
- Builds the frontend React app with Vite and packages it into a minimal Nginx Alpine image.
- Starts both containers on a shared internal Docker network.

Service	Exposed Port	Description
frontend	80	Nginx serves the React app and reverse-proxies API and socket traffic.
backend	5001	Express REST API and Socket.io server.

The Nginx configuration in `frontend/nginx.conf` handles single-page-application fallback routing and transparently proxies all requests under `/api/` and `/socket.io/` through to the backend container, so the frontend and backend appear as a single origin to the browser.

13. Cloud Deployment

13.1 Frontend on Vercel

The frontend deploys to Vercel by importing the GitHub repository and setting the project's root directory to frontend. Vercel auto-detects the Vite build and runs `npm run build`; a bundled `vercel.json` ensures React Router's client-side routes fall back correctly instead of returning 404s on refresh.

13.2 Backend on Railway or Render

The backend deploys to either Railway or Render by connecting the same GitHub repository, setting the service's root directory to backend, and configuring the start command as `node server.js`. All environment variables listed in Section 11 must be added to the hosting provider's dashboard. A bundled `railway.json` preconfigures the correct build and start commands for Railway specifically.

13.3 Database

For production, a serverless PostgreSQL provider such as Neon or Supabase is recommended. After provisioning a database, its connection string is set as the `DATABASE_URL` environment variable on the backend deployment, and the schema is pushed with:

```
npx prisma db push
```

14. Security Considerations

Security is addressed at several layers of the stack rather than in a single place. Passwords are never stored in plain text — they are hashed with bcrypt before being written to the database, and the plain-text value is discarded immediately after hashing. Session state is stateless, carried in a signed JWT rather than a server-side session store, which simplifies horizontal scaling of the backend.

Every protected route is guarded by the two-layer middleware described in Section 9: token verification followed by role authorization. This defense-in-depth approach means that even if a UI element were shown to a user in error, the API itself would still reject an unauthorized action. Helmet is used to apply secure default HTTP headers, and Google OAuth offers users a sign-in path that avoids handling a password at all.

File uploads are routed through Cloudinary rather than stored directly on the application server, reducing the attack surface associated with handling arbitrary user-uploaded files. Invitation links are tokenized and time-limited, preventing them from being reused indefinitely or guessed.

15. Testing and Quality Assurance

Correctness in a system like ProjectDock matters most at the boundaries between its layers: between the frontend and the API, between the API and the database, and between the REST layer and the real-time Socket.io layer that must stay consistent with it. A meaningful testing strategy for the platform focuses on these boundaries rather than treating every layer as independent.

15.1 Backend Testing

At the API level, testing centers on the controllers and middleware that make up each route: verifying that authentication middleware correctly rejects requests without a valid JWT, that `authorizeRoles` correctly blocks a role that is not on a route's approved list, and that each controller produces the expected database state through Prisma for both successful and failing inputs. Particular attention is warranted around the join-table models — `ProjectMember`, `TeamMember`, and `TaskAssignee` — since incorrect handling of these relationships is the most likely source of a user seeing data they should not have access to.

15.2 Frontend Testing

On the frontend, testing focuses on the `PermissionGate` component's rendering logic across each role, the correctness of React Query cache invalidation after a mutation (so that, for example, moving a task on the Kanban board is reflected in the global task list without a manual refresh), and the behavior of the Socket.io client context when a connection drops and needs to reconnect without losing the user's place in the application.

15.3 Manual and Exploratory Testing

Given the real-time nature of the platform, a portion of quality assurance is best done manually with two or more browser sessions open simultaneously — one making a change, such as commenting on a task or sending a team chat message, and the other observing whether that change arrives instantly and

renders correctly. This kind of multi-client exploratory testing catches timing and ordering issues that are difficult to reproduce in an automated single-client test.

16. Risk Assessment and Limitations

As with any system, ProjectDock's current design carries a number of known limitations and risk areas worth documenting explicitly, both to set expectations and to guide future hardening work.

16.1 Scalability Considerations

The Socket.io server, as currently structured, holds real-time connection state in the memory of a single backend process. Running multiple backend instances behind a load balancer for horizontal scaling would require introducing a shared adapter — typically backed by Redis — so that an event emitted by one instance reaches clients connected to another. This is a well-understood extension of the current architecture, but it is not yet in place and should be treated as a prerequisite for scaling the platform beyond a single backend node.

16.2 Data Integrity Under Concurrent Edits

Because multiple users can edit the same task, wiki page, or board configuration at nearly the same moment, the system is exposed to last-write-wins behavior at the database layer unless explicit optimistic locking or field-level merge logic is introduced. For most fields this is an acceptable trade-off, but for higher-stakes fields such as task status transitions on a board with strict workflow rules, additional conflict handling would reduce the chance of a user's change being silently overwritten.

16.3 Dependency on Third-Party Services

File uploads depend on Cloudinary and outgoing email depends on an SMTP provider configured through Nodemailer. An outage or misconfiguration of either service degrades specific features — attachments or invitations, respectively — without necessarily affecting the rest of the platform, but both represent external dependencies that are outside the direct control of the ProjectDock codebase and should be monitored accordingly in a production deployment.

16.4 Secret and Credential Management

The current setup relies on environment variables for all secrets, including the JWT signing key and third-party API credentials. This is a reasonable baseline, but as the platform moves toward a production deployment with multiple environments, adopting a dedicated secrets manager rather than plain .env files reduces the risk of credentials being committed to version control or leaked through logging.

17. Glossary of Terms

Term	Definition
JWT	JSON Web Token — a compact, signed token used to represent an authenticated session.

Term	Definition
ORM	Object-Relational Mapper — a library that maps database rows to application objects; Prisma is ProjectDock's ORM.
RBAC	Role-Based Access Control — restricting actions based on a user's assigned role.
Kanban	A visual board method for managing work as it moves through columns representing stages.
Sprint	A fixed time period during which a defined set of work is completed.
Gantt Chart	A bar-chart style view showing tasks against a timeline, including duration and dependencies.
WebSocket	A persistent, two-way network connection used here via Socket.io for real-time updates.
SPA	Single-Page Application — a web app that loads once and updates content dynamically without full page reloads.
OAuth	An open standard for delegated authentication, used here for Google sign-in.
Monorepo	A single repository containing multiple related applications or packages.

18. Conclusion and Future Scope

ProjectDock demonstrates a complete, modern approach to project management software, combining task tracking, sprint planning, timeline visualization, team collaboration, time logging, and analytics into a single, cohesively designed platform. Its architecture — a React frontend, an Express and Prisma backend, and a PostgreSQL database, connected by REST and WebSocket channels — provides a solid, extensible foundation for further development.

Natural directions for future work include deeper integrations with external developer tools such as GitHub or GitLab for automatic task linking, richer automation rules for moving tasks between states, mobile applications built on the existing API, and expanded reporting capable of exporting to formats such as PDF or CSV for stakeholders outside the platform.

Overall, ProjectDock succeeds in bringing the core responsibilities of software project management into one connected, real-time system, reducing the tool-switching overhead that many teams currently accept as a cost of doing business.

Appendix A: Complete Feature List

- Project lifecycle management with Active, On Hold, Completed, and Archived statuses
- Rich project metadata: name, key, description, logo, banner, color theme, priority, visibility, tags, budget
- Project dashboard with task, member, and progress summary cards
- Favorite and bookmark projects
- Task creation with priority, status, assignees, labels, due dates, and estimated hours
- Drag-and-drop Kanban board with customizable columns and swimlanes
- Task detail view with comments, attachments, and activity history
- Global, filterable task list across all projects
- Sprint creation, velocity tracking, and automatic task carry-over
- Gantt-style timeline with durations, dependencies, and progress
- Team creation with leads, members, and role assignment
- Real-time team chat and activity feed
- Individual and team-based project membership assignment
- Role-based permission checks at the API and UI level
- Organization-level analytics dashboard with interactive charts
- Time-logged-per-user and overdue task reporting
- Live timer and manual time entry with daily/weekly summaries
- Project-scoped Wiki with a rich text editor
- Threaded project discussions with nested replies
- Project file storage and attachments
- Real-time in-app notifications with a topbar badge
- Tokenized email invitations for organizations and projects
- Profile, avatar, and password management
- Light and dark mode with persisted preference
- Native email/password authentication and Google OAuth 2.0 sign-in

References

Source Repository: github.com/Vikashh08/Project-Management-SaaS

React Documentation: react.dev

Prisma Documentation: prisma.io/docs

Socket.io Documentation: socket.io/docs

Express Documentation: expressjs.com